The Bridge Experiment: Does an Informal↔Formal Join Move End-Task Lean Performance?

Jack McCarthy
WikiLean · `https://wikilean.jackmccarthy.org`
with experiments executed by Claude Code agents under direction

Final report — July 25, 2026

Preregistration: `docs/research/BRIDGE-EXPERIMENT.md`,
commit `0d36f2664ab2ebb2c7b80b350d3c9bd820414335` (2026-07-16).

Results log: `docs/research/BRIDGE-RESULTS.md`. All data, code, and run transcripts are preserved in the private repository `Deicyde/wikilean-bridge-experiment` (file map in Section 8). Every number below is recomputable from a file in that repository; the file is named where each table appears.

1 Abstract

We test whether making the informal↔formal join easy — a curated dictionary between mathematical concepts and Mathlib declarations (the WikiLean "Brain"), as opposed to mere access to both corpora — improves language-model performance on formalization tasks. Five preregistered arms isolate the join: no tools (A), informal search (B), formal search (C), the joined bridge (D), and both corpora side by side but unjoined (E). Grading is mechanical throughout, and typechecking alone never counts as success. We report three headline results. First, on 100 contamination-proof tasks built from Mathlib theorems newer than every index, the bridge arm reaches 42% folded success against 16% for the unjoined-tools control (McNemar 32 vs. 6 discordant, exact $p < 0.0001$), while the no-tool arm collapses from 59.6% to 20%, which puts a number on ProofNet memorization; the bridge also cuts hallucinated-decl citations to 6.8%, against 17.7–26.3% in every other arm. Second, on third-party retrieval gold (MathlibQR-810, MathlibMPR), a Sonnet agent given the union of bridge and formal-search tools, plus a manual distilled from measured tool behavior, sets the best rows we know of on both benchmarks (R@10 0.885; group-recall@10 0.557 against the specialist SOTA of 0.461). Third, the join does not help premise selection (0.272 vs. 0.453 for formal search); this is the concept≠premise boundary we preregistered, now quantified as an 18pp API target.

2 Hypothesis and preregistered design

Hypothesis (operationalized). Since "necessary" is unprovable, the preregistration commits to three predictions. P1: an agent with the *joined* dictionary beats an agent given informal and formal search *separately* — if D > E, then the join, not corpus access, carries the effect. P2: the bridge solves at lower cost. P3: the bridge lifts grounding (fewer hallucinated declaration citations), not just compile rate.

The five arms. All five arms share the same model, prompt, and budgets within each phase; the only thing that differs between them is the tool manifest (`bench/run_bridge.py`, manifests in `bench/arms/`); see Table 1.

Models per phase. Tier 1 (statement autoformalization) uses `claude-haiku-4-5` (model id `claude-haiku-4-5-20251001`) in all five arms. Bridge v2 (retrieval and proving) uses `claude-sonnet-5` in all agent arms, per an explicit pre-execution decision (`docs/research/BRIDGE-V2-BENCHMARKS.md`, 2026-07-25).

Arm	Tools	Isolates
A `no_tools`	none (`-tools ""`)	floor / memorization
B `informal`	Wikipedia + nLab search/fetch, no Lean mapping	informal reasoning alone
C `formal`	loogle + ripgrep + source read over pinned Mathlib	the LeanSearch-class status quo
D `wikibrain`	the Brain MCP (`brain_bridge/search/cell/transfer/neighborhood/snippets/filter` + batch `decl_exists`)	the join
E `B+C unjoined`	B's tools AND C's tools, no join	the decisive control

Table 1: The five preregistered arms. Identical model, prompt, and budgets; only the tool manifest differs.

Grading rules. Typecheck is never success by itself; TheoremGraph [1] reports 22/24 outputs typechecking while only 5/24 were correct. A Tier-1 run counts as a success only when it produced a declaration, cited zero hallucinated names, and typechecks on the pinned toolchain. We quarantined the preregistered LLM-judge faithfulness leg behind a human-calibration gate and ultimately dropped it in favor of graders that existed before the arms did — third-party gold labels and the Lean kernel; Section 7 explains why. Hallucinated-decl rate is graded against a union oracle (doc-gen4 declaration data ∪ verified renames; the extractor is documented in `bench/score_bridge.py`).

Task sets. Tier 1a is ProofNet#: 371 problems (341 eval plus 30 prompt-freezing dev, all scored), pinned to `PAug/ProofNetSharp` `@a8da405fbd1e348a87445c2e562c747b7e26dc8f` (MIT; [12]) and graded on Lean v4.32.0-rc1 / Mathlib `a33a5ccd`. Tier 1b is 100 fresh tasks built from theorems merged into Mathlib master between 2026-07-03 and 07-16, provably absent from every arm's index including the Brain's decl universe (`bench/data/fresh_tasks.stats.json` records the held-out guarantee); these are graded on Lean v4.33.0-rc1 / Mathlib `9944fe29`. A second independent determinacy annotation (agreement 79%, $\kappa \approx 0.20$; the annotators apply complementary strictness) defines a 74-task both-determinate primary subset.

Budgets. Every arm ran under the same budget: 30 turns stated in the prompt and a 10-minute wall clock, enforced. Agents had no in-loop typechecker and were instructed to verify cited names through their tools.

Protocol deviations (recorded in the preregistration before any result existed; numbering as in that document):

1. The turn budget is prompt-stated and wall-clock-enforced (the CLI lacks `-max-turns`); turns are recorded per row.
2. Arm D talks to the production Worker code served locally over the shipped shard bytes (`bench/arms/local_worker.mts`), with snapshot pins echoed per response.
3. Grading uses a persistent REPL (Mathlib loaded once) rather than single-shot elaboration, on the same pins.
4. Agents had no in-loop typechecking this campaign, which makes hallucinated-decl rate a pure grounding measure.
5. Judge grades were excluded from all claims pending human calibration (subsequently superseded by the v2 judge-free design).
6. **Skill-leak seal (deviation 6).** The dev smoke showed tool arms B–E calling the CLI's built-in `Skill`/`ToolSearch` (32–56 calls/arm) and loading, for example, Mathlib-conventions text that arm A structurally could not reach — an asymmetric contamination. We added `Skill`, `ToolSearch`, and `Agent` to the disallowed-tools list *before any eval run*, quarantined the 120 contaminated B–E dev runs unscored to `bench/data/runs_devleak_2026-07-18/`, and re-ran them clean.
7. **Trace telemetry (deviation 7).** From eval arm C onward we retained per-call tool traces (truncated input, result head, error flag), observation-only — nothing any agent sees changed. As a consequence, eval arms C/D/E and all fresh runs carry traces; eval A/B carry name-counts only.

3 Tier 1: statement autoformalization (claude-haiku-4-5)

3.1 Tier 1a — ProofNet# ($n = 371$/arm)

The source is `docs/research/BRIDGE-RESULTS.md`, and every number is recomputable from `bench/data/bridge_summary.json` plus `bench/data/runs/`. Success requires a produced declaration, no hallucinated citation, and a passing typecheck. Results in Table 2.

metric	A none	B wiki	C formal	D wikibrain	E B+C unjoined
success (folded)	59.6%	57.1%	62.3%	**64.2%**[a]	60.9%
success_proxy (no-halluc)	76.8%	72.8%	72.2%	**84.6%**	71.2%
typecheck ok	66.6%	63.6%	81.9%	74.7%	**83.3%**
halluc-decl rate	10.1%	11.0%	10.6%[a]	**5.9%**	11.3%
runs w/ hallucination	86	101	103	**57**	107

[a] Corrected for display rounding relative to the markdown report and results log, which printed D success 64.1% and C halluc-decl rate 10.7%: the exact recomputed values are $238/371 = 64.15\%$ and $140/1315 = 10.65\%$. No claim is affected.

Table 2: Tier 1a — ProofNet# ($n = 371$/arm). Bold marks the best column per row.

The preregistered McNemar test of D against E gives 63 vs. 51 discordant pairs, exact $p = 0.30$: the direction favors D, but the test is underpowered at this effect size. Two other readings matter. Arm A scoring 59.6% with zero tools is substantial ProofNet memorization, and was the motivation for Tier 1b. And the typecheck row *anti-correlates* with grounding — E leads on typecheck (83.3%) while trailing on hallucinations, reproducing TheoremGraph's typecheck-is-not-a-signal finding at 15× their n.

3.2 Tier 1b — the fresh set ($n = 100$/arm, contamination-proof)

Same files, restricted to the fresh tasks. With memorization stripped, the field reorders (Table 3, Figures 1 and 2).

metric	A none	B wiki	C formal	D wikibrain	E B+C unjoined
success (folded)	20%	22%	25%	**42%**	16%
halluc-decl rate	21.2%	17.7%	20.9%	**6.8%**	26.3%
runs w/ hallucination	54	48	49	**23**	36

Table 3: Tier 1b — the fresh set ($n = 100$/arm), built from Mathlib theorems newer than every arm's index.

Here the preregistered hypothesis test is decisive. McNemar D-vs-E gives **32 vs. 6 discordant pairs, exact** $p = 2.4 \times 10^{-5}$; D-vs-C gives 28 vs. 11 ($p = 0.0095$) and D-vs-A 29 vs. 7 ($p = 0.0003$). On the held-out set it is the *join* that carries the effect, not tool volume. The result also survives restriction to the 74-task both-determinate primary subset (recomputed from the `bench/data/fresh_tasks.jsonl` det fields crossed with the paired matrix): D solves 31/74 (41.9%) against E's 14/74 (18.9%), McNemar 22 vs. 5, exact $p = 0.0015$, so the effect is not an artifact of underdetermined prompts.

Three secondary observations. Arm A collapses from 59.6% to 20% off-distribution, which quantifies the memorization. D's hallucination advantage *widens* off-distribution (6.8% vs. 17.7–26.3%). And arm E is the worst arm at 16%, producing no declaration at all in 31 of its 100 fresh runs — unjoined tool volume can be actively harmful.

Cost tells the same story (recomputed from run rows; mean per task, folded over all 471 runs/arm): A \$0.034, B \$0.048, C \$0.140, **D \$0.121**, E \$0.128, with mean wall-clock C 116 s, D 89 s, E 106 s. The bridge arm outscores the cheaper-per-task C and E while costing less than either, so P2 holds directionally.

3.3 Trace attribution (deviation-7 telemetry, eval C/D/E)

The traces (`bench/trace_analysis.py` over `bench/data/runs/`, eval split) let us ask where cited names actually came from. Between 98% and 100% of traced tool-arm runs cite at least one declaration that visibly

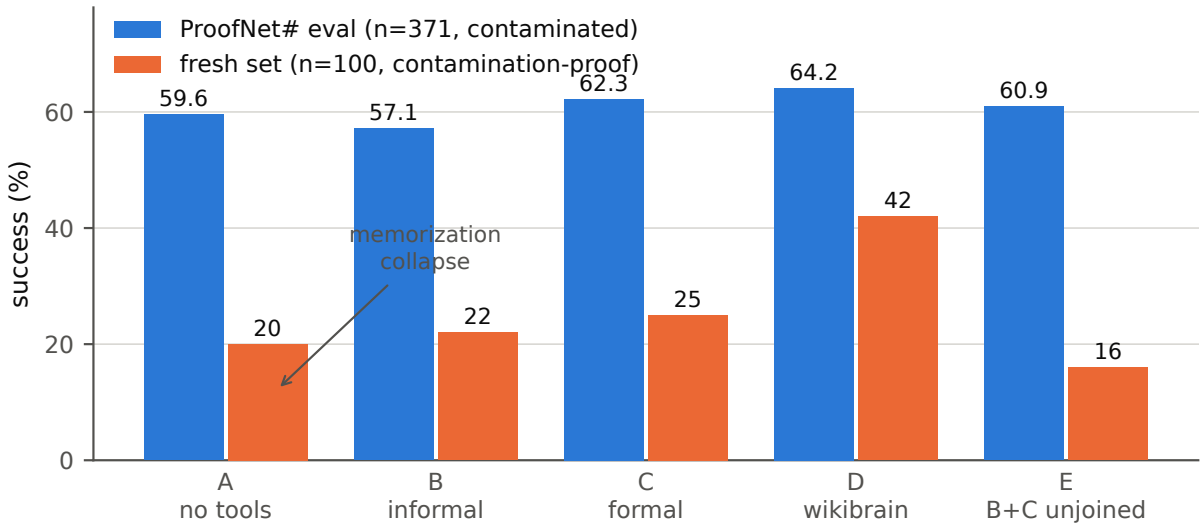

Figure 1: Tier-1 folded success by arm, ProofNet# eval versus the contamination-proof fresh set. Off-distribution, the no-tool arm collapses 59.6% → 20% while the bridge arm (D) holds 42% against 16% for the unjoined-tools control (E). Data: `bench/data/bridge_summary.json` (paired matrix, split by task id).

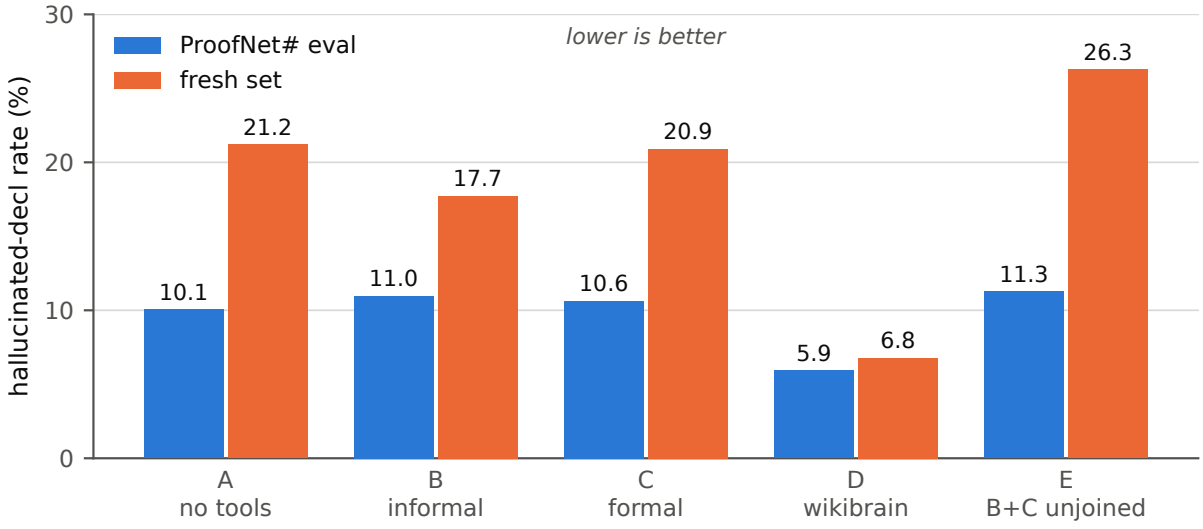

Figure 2: Hallucinated-declaration citation rate by arm (share of cited declaration names absent from the union oracle). D's grounding advantage *widens* off-distribution: 6.8% on the fresh set versus 17.7–26.3% for every other arm. Data: recount over `bench/data/runs/` with the `bench/score_bridge.py` extractor and oracle.

surfaced in a tool result, and only 10–17% of citations never touched the tools. In arm D, **35% of runs (116/329) cite a declaration that came out of a `brain_bridge` / `brain_transfer` result** — English in, formal name out — and this is an undercount, since result heads truncate at 200 chars. The sharpest signal is among *hallucinated* citations: the checked-and-cited-anyway rate is 35% for C and 30% for E but only **13% for D**. Binary `decl_exists` verification disciplines the model where the fuzzy search neighborhoods of loogle and grep let it fool itself. This is the mechanism behind the hallucination rows above.

4 Third-party retrieval: MathlibQR-810 and MathlibMPR (claude-sonnet-5)

Tier-1's remaining faithfulness leg depended on an LLM judge whose error could correlate with arm, so Bridge v2 replaces judge-dependent grading with graders that predate the arms: third-party expert gold labels here, and the Lean kernel in Section 5. The data come from `frenzymath/LeanSearch-v2` @`94f4888cbaf9` (CC BY 4.0; LeanSearch-v2 [3]), copied under `bench/v2/data/`. There are four arms: N (no tools), F (`loogle` + `decl_grep` + `decl_read`), W (the Brain MCP), and WF (W ∪ F **plus** `bench/v2/AGENT_MANUAL.md` prepended to the prompt — the manual is deliberately part of the condition; see the confound note below). Scoring is `bench/v2/score_retrieval.py`, exact full-name match only; run rows with full gzipped stream-json transcripts live in `bench/v2/runs/agent/`.

4.1 MathlibQR fair-810 — concept retrieval (810 expert query rows, 6 styles)

system	R@10	nDCG@10
published: TheoremGraph	0.775	0.548
published: LSv2 retriever+reranker	0.780	0.623
system-mode wikibrain (one `brain_bridge` call, no LLM)	0.036	0.031
agent N	0.633	0.598
agent F	0.831	0.790
agent W	0.816	0.781
agent WF	**0.885**	**0.839**

Table 4: MathlibQR fair-810 concept retrieval. Exact full-name match; gold is one declaration per query row.

The first thing Table 4 shows is a deficiency. System mode — one `brain_bridge` call, no LLM — scores 0.036, because the API's free-text entry is a label/alias resolver, not a semantic retriever: nickname queries reach nDCG 0.196 while the descriptive styles (LaTeX, natural-language, special-case) score 0.000. Yet an agent *iterating* over the same API reaches 0.816. The content is in the graph; the single-shot entry point is the gap. This is the headline API deficiency.

Among the agent arms, F and W are statistically tied (paired discordants 78 F-only vs. 66 W-only, exact $p = 0.36$), and both land nominally above the published retriever rows — with the apples-to-oranges caveat that agents reason and verify while retrievers embed once (Section 7). The two toolkits have different textures: W wins the special_case style (nDCG 0.523 vs. F's 0.384 — the Brain's curated special_case bonds are the signal), while F wins the Lean-syntax styles. Grep cannot find what source text never names.

WF is decisive. It beats F (71 vs. 27 discordant, exact $p = 1.0 \times 10^{-5}$) and W (83 vs. 27, $p = 8.3 \times 10^{-8}$), and sits +10.5pp R@10 above the best published retriever row. The style spread narrows as well: every style reaches ≥ 0.55 nDCG, and five of six reach ≥ 0.81. Agent N's 0.633, by contrast, includes 143/810 rows scored 0 for format non-compliance (no JSON array in the final message), an agent-mode failure the manual explicitly addresses.

The tool census: F averages 2.2 calls/run and W 3.5 (`decl_exists` 1,251 + `brain_bridge` 608 — a verify-then-cite pattern), while WF averages 4.6 calls pooled across both benchmarks in a genuine dual-toolkit mix (`decl_grep` ∼1.5k, `decl_exists` ∼0.9k, `brain_bridge` ∼0.7k, `decl_read` ∼0.5k, `loogle` ∼0.4k). The `brain_cell` misuse pattern, a 63% failure rate in the Tier-1 traces, disappears under the manual.

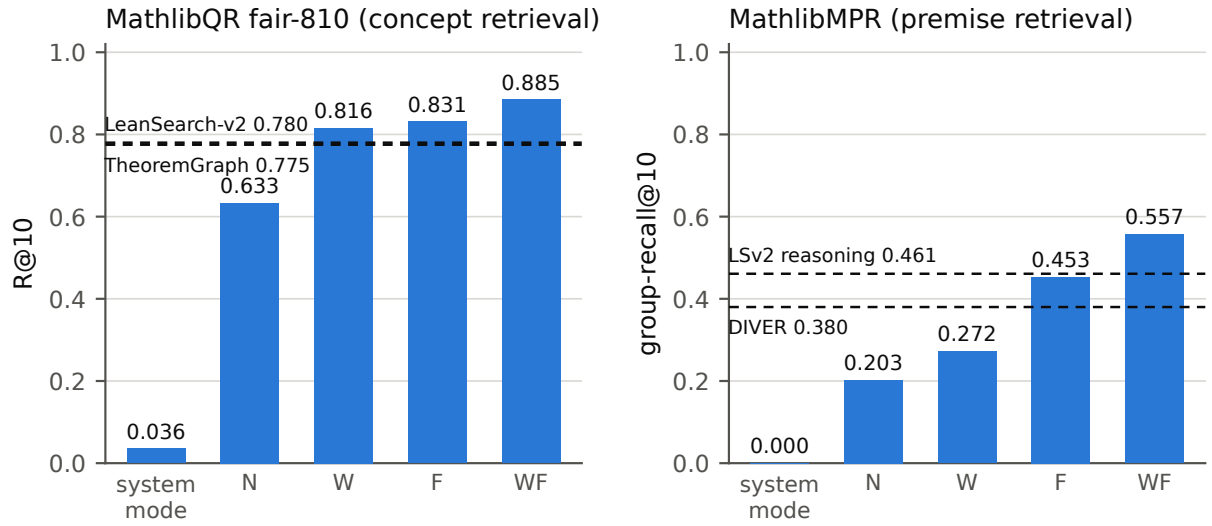

Figure 3: Retrieval by system, with published anchors as dashed reference lines. Left: MathlibQR fair-810 R@10 (anchors: TheoremGraph 0.775, LeanSearch-v2 retriever+reranker 0.780). Right: MathlibMPR group-recall@10 (anchors: LeanSearch-v2 reasoning 0.461, DIVER 0.380). System-mode is a single `brain_bridge` call with no LLM; N/W/F/WF are Sonnet agent arms. Data: rerun of `bench/v2/score_retrieval.py`.

4.2 MathlibMPR — premise retrieval (69 post-cutoff PR theorems, expert premise groups)

system	group-recall@10
published: LeanSearch-v2 reasoning	0.461
published: DIVER	0.380
published: TheoremGraph concept-tuned	0.165
system-mode wikibrain	0.000
agent N	0.203
agent W	0.272
agent F	0.453
agent WF	**0.557**

Table 5: MathlibMPR premise retrieval: a group is covered when any of its interchangeable members appears in the top 10.

Two results stand out. A generic Sonnet agent with grep and loogle **matches the specialist premise retriever** (0.453 vs. 0.461), a notable baseline result on its own. The Brain, by contrast, helps over memory (+7pp) but trails formal search by 18pp: this is the preregistered concept ≠ premise boundary (TheoremGraph's negative transfer), now measured on our own tools, and it gives `brain_premises` (BRIDGE-ISSUES #7) a quantified 18pp target. WF again beats both the best single arm (+10pp over F) and the published SOTA on its own benchmark (0.557 vs. 0.461).

The WF confound, stated plainly. WF is union tools plus the evidence-based manual, and the two are not separated in this campaign. A bare-union ablation is one flag away (`MANUAL_ARMS` in `bench/v2/run_agent.py`) and was not run, so WF results should be read as "a maximally equipped, briefed agent", not as a clean marginal effect of tool union.

5 SorryDB: kernel-graded proving in live repositories (claude-sonnet-5)

SorryDB [5] serves unsolved `sorry`s from real Lean repositories, and success is defined by the repo *building* with your proof: no judge exists anywhere in the loop, and no solutions exist to memorize. Our snapshot is the SorryDB-2601 1,000-task evaluation split (`bench/v2/data/sorrydb/SorryDB_2601_1000_evaluation_split.json`).

Subset construction and the snapshot-rot finding. Disk permitted one repository build at a time, so we sampled repositories rather than tasks: every task of a chosen repository is included, and verification cost amortizes across it. The original freeze — 8 repositories, 164 tasks, chosen for toolchain availability, build weight, and domain diversity, plus one pedagogical floor — lost 74 of 164 tasks (45%) in the roughly six months since the snapshot: the pinned commits of GlimpseOfLean, LeanCourse25, and most Foundation and SemicircleLaw tasks no longer exist on GitHub. When a repository's history is rewritten, commits no branch reaches are eventually deleted, and a snapshot's pins die with them. We could not retrieve these commits by any route (direct git fetch by hash, GitHub's archive downloads, or the authenticated API). Detection is the subtle part: the archive endpoint answers preflight HEAD requests for dead commits with HTTP 200, so a liveness check that does not attempt a real fetch reports the pins as healthy. We record this as a methodological finding: benchmarks pinned to live repositories rot at the commit level, and the rot is invisible unless every pin is actually re-fetched before use. The refreeze (`bench/v2/sorrydb_prep.py`, commit `db679e8d`) keeps only verified-fetchable pins: 171 tasks across 10 repositories (PrimeNumberTheoremAnd 21, curve25519-dalek-lean-verify 21, category-theory-in-context 21, PersistentDecomp 20, brownian-motion 20, graphiti 20, LeanAPAP 20, MiscYD 20, SemicircleLaw 7, Foundation 1).

Protocol. The agent phase is build-free and one-shot. Each row carries the goal state plus a ±60-line file window at the pinned commit; the agent (600 s timeout, arms N / F / WF as in Section 4) outputs one replacement for the `sorry`, under an explicit honest-abstention rule (output the literal `sorry`). Verification (`bench/v2/verify_sorrydb.py`) clones each repo at its pin, establishes a pristine baseline build, splices each candidate over the span (asserted to literally read `sorry`), and rebuilds the module; success requires exit 0 with no `sorry`/axiom warning.

Results — recomputed for this report from `bench/v2/runs/sorrydb/verify.jsonl` plus the run rows (script logic in Section 8; $n = 171$ tasks/arm) — in Table 6 and Figure 4.

	N no tools	F formal	WF union+manual
run rows present	168	169	171
no output (timeout / no lean block)	70	15	11
honest give-up (literal `sorry`)	40	83	86
candidate proofs submitted	58	71	74
proved (kernel)	**2**	**9**	**10**
failed	48	54	56
unspliceable	5	6	5
no verdict (see note)	3	2	3
proved / 171 tasks	**1.2%**	**5.3%**	**5.8%**
proved / candidates	3.4%	12.7%	13.5%
total cost (USD)	$58.97	$102.76	$108.87
cost per proved	**$29.48**	**$11.42**	**$10.89**
mean wall-clock / task	120 s	198 s	183 s

Table 6: SorryDB kernel-graded proving, one-shot with no compiler in the loop ($n = 171$ live-pin tasks/arm).

Three notes complete the record. (i) 8 candidate proofs — all from curve25519-dalek-lean-verify, whose verification pass covered only 2 of its 10 candidates — carry no kernel verdict in `verify.jsonl`; they are

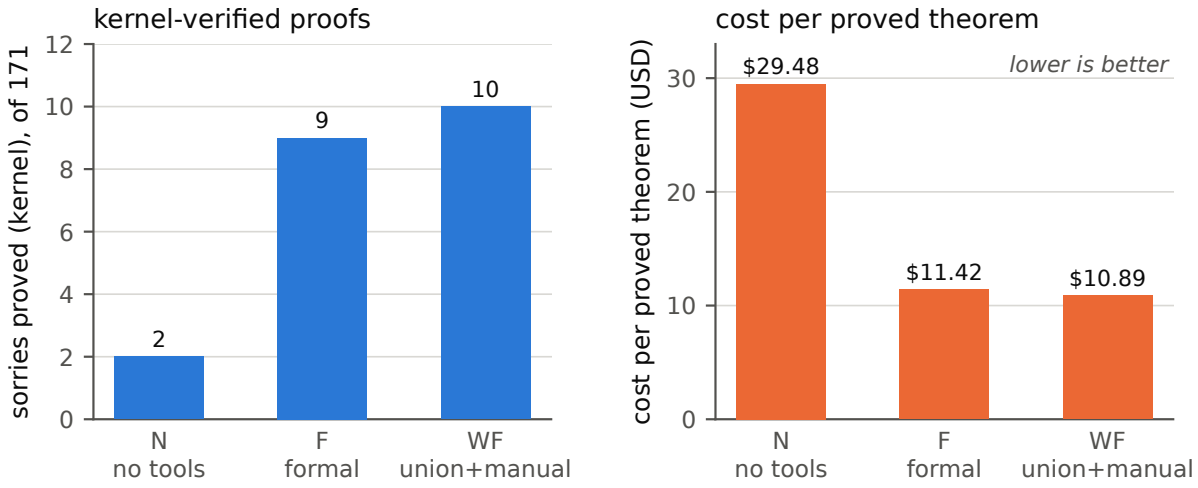

Figure 4: SorryDB, one-shot drafting with no compiler in the loop. Left: kernel-verified proofs out of 171 live-pin tasks. Right: cost per proved theorem — tools raise total spend but *cut* the cost of each success by 2.7×. Data: `bench/v2/runs/sorrydb/verify.jsonl` × the run rows.

conservatively counted as not-proved above. (ii) 3 (N) and 2 (F) tasks have no run row at all (runner losses). (iii) 2 stale verdicts against dead-pin rows from the first verification pass are excluded. Distinct sorries proved across all arms: 13; N's proofs are a strict subset of both F's and WF's; WF vs. F overlap 6, WF-only 4, F-only 3 — no significance claim at this n.

Reading. Tools triple-to-quintuple the kernel-verified prove rate and nearly triple the honest-abstention rate; N instead burns its budget, with 70 of its 168 rows producing nothing, mostly 600 s timeouts. Cost-per-proved-theorem *falls* by 2.7× even as total tool spend rises — verification effort is cheaper than blind search. One comparison caveat: the published SorryDB agentic best (≈30%, union 35.7%) is not comparable, both because the task subset differs (ours is weighted toward research-frontier repos: PNT+, a cryptographic verification codebase, LeanAPAP) and because the protocol differs materially (published agents iterate against the build; ours drafted one-shot with no compiler in the loop).

6 Synthesis

What the join adds. Two things, both now measured twice. The first is *grounding*: binary existence verification at the informal→formal boundary collapses hallucinated citations (5.9%/6.8% vs. 10–26%; checked-and-cited-anyway 13% vs. 30–35%), and the effect grows off-distribution, where memory fails. The second is *concept entry*: when the query is a description rather than a name — the fresh set, the special_case query style — the curated join finds what lexical and type-pattern search cannot (42% vs. 16–25%; nDCG 0.523 vs. 0.384). The fresh-set McNemar ($p < 0.0001$) says the join itself, not corpus access, carries this.

Where it does not. Premise selection ranks by a different signal than concept identity: W trails F by 18pp on MathlibMPR, exactly as preregistered from TheoremGraph's negative transfer. And the API's single-shot free-text entry is a label resolver (0.036 R@10 in system mode); everything the agent arms recover, they recover by reformulating against it.

The composition result. The toolkits are complementary, and a briefed agent composes them into the best rows we know of on both third-party benchmarks (QR 0.885, MPR 0.557) — on gold labels we did not write. This is the Bridge thesis restated in retrieval form: the join *plus* the territory beats either alone.

API roadmap, derived. Four items follow directly from the data. First, a semantic statement-level index behind `brain_bridge`: the 0.036 → 0.816 single-shot/agent gap is an entry-point problem, not a content problem. Second, `brain_premises(goal_state)`, a premise-level ranking mode with a measured 18pp target (BRIDGE-ISSUES #7). Third, the eight pre-campaign API requirements (batch `decl_exists`, signatures + imports per hit, generalization surfacing, honest abstention, cursored neighborhoods, snapshot echo, the composite `brain_bridge`, teaching tool descriptions) were implemented before the campaign and are what the arms exercised. Fourth, the `AGENT_MANUAL.md` pattern itself: distilling measured tool behavior (∼5,500 logged calls) into a briefing eliminated the dominant misuse mode (`brain_cell` 63% → ∼0) and is the cheapest intervention in the whole study.

Cost economics. In Tier-1 the bridge arm is cheaper *and* better than its tool-bearing competitors ($0.121/task vs. C $0.140, E $0.128, at +2–26pp success). Retrieval runs at $0.08–0.21/query (QR) and $0.18–0.44/query (MPR). In proving, tool arms cut cost-per-proved-theorem from $29 to $11 while raising the prove rate 5×. Across all three phases, adding the join never made the task more expensive per success.

7 Limitations and threats to validity

1. **One model per phase.** Tier-1 is Haiku-only (per-model run dirs were not keyed — fixed in v2, where dirs are (benchmark, arm, model)-keyed) and v2 is Sonnet-only, so a bridge that only helps some capability band would not be distinguished. The preregistered two-model grid remains undone.
2. **Agent-vs-retriever comparability.** Our QR/MPR agent rows spend multi-call reasoning and verification per query; published retriever rows embed once. "Above the published rows" is a statement about achievable quality at agent cost, not a same-budget comparison. System-mode is the same-budget comparison, and it loses badly.
3. **The WF manual confound.** WF bundles tool union with the briefing; the ablation exists in the harness but was not run.
4. **Power on secondary contrasts.** F-vs-W on QR is a null at $n = 810$ ($p = 0.36$); MPR has $n = 69$; SorryDB proved counts are single digits per arm — overlap patterns there are descriptive only.
5. **The judge leg was dropped, not repaired.** The preregistered faithful@budget metric (BEq+ equivalence) was never graded: the judge required human calibration (deviation 5), Jack's arm-correlated-bias critique stood, and the field precedent (TheoremGraph rejecting its own judge as over-generous) argued against resting headline claims on one. The v2 replacement — graders that predate the arms — is stronger, but it means Tier-1 "success" can still reward a well-formed wrong statement; the Tier-1 fresh-set numbers are best read jointly with the hallucination and trace evidence.
6. **Tier-1a is contaminated by construction** (arm A: 59.6%); we report it for the paired deltas and as the memorization control against Tier-1b, never alone.
7. **SorryDB caveats.** One-shot drafting without a compiler under-represents every arm relative to published loops; the subset is repo-sampled, not difficulty-matched; 8 candidates lack verdicts; and 45% snapshot rot means the *original* frozen subset is already partly unreproducible upstream — the preserved `tasks_frozen.jsonl` (goal states + context windows) is the durable record.
8. **Mechanical extractors.** Cited-declaration extraction is a documented regex heuristic; hallucination rates are comparable across arms (same extractor) but not exact.

8 Data availability

Everything needed to recompute this report lives in the private preservation repository `Deicyde/wikilean-bridge-experiment` (report at the root as `README.md` and under `report/`). Provenance commits refer to the WikiLean working repository. See Table 7.

Recomputation entry points: `bench/score_bridge.py` (Tier-1), `bench/trace_analysis.py` (Section 3.3), `bench/v2/score_retrieval.py` (Section 4 — reprints every table), `bench/v2/verify_sorrydb.py` (Section 5 verdicts; the Table 6 rows are a straight aggregation of `verify.jsonl` × the run rows, filtered to

Path (preservation repo)	Contents	Key provenance commits (WikiLean)
`docs/research/BRIDGE-EXPERIMENT.md`	preregistration incl. deviations 1–7	`0d36f266` (2026-07-16)
`docs/research/BRIDGE-RESULTS.md`	Tier-1 + retrieval results log	`53f5cb31`, `daac6107`
`docs/research/BRIDGE-V2-BENCHMARKS.md`	v2 design + verified sources	—
`docs/research/BRIDGE-ISSUES.md`	issue queue / history	`cdc2d743`
`bench/*.py`, `bench/README.md`, `bench/arms/`	Tier-1 harness: runner, scorer, REPL typecheck rig, trace analysis, arm manifests	`068abe34`, `e554902e`, `6c2ca704`, `31b95caa`, `79ac3dfe`
`bench/data/bridge_tasks.jsonl` (+`.stats`)	371 ProofNet# tasks; source pin + MIT licence in stats	`e554902e`, `ec2a9e11`
`bench/data/fresh_tasks.jsonl` (+`.stats`)	100 fresh tasks + determinacy annotations + held-out guarantee	`df5dbf92`, `a0d45103`
`bench/data/gold_census.json`, `fresh_census.json`	which golds elaborate on which pin; fresh pin = Lean v4.33.0-rc1 / Mathlib `9944fe29` (Tier-1a pin = v4.32.0-rc1 / `a33a5ccd`, recorded in `bench/README.md`)	`01e8612b`, `185377cc`
`bench/data/bridge_summary.json`	Tier-1 scored summary: per-arm aggregates, paired matrix, McNemar tables	`53f5cb31`
`bench/data/runs/`	all 2,355 Tier-1 run rows (5 arms × 471)	—
`bench/data/runs_devleak_2026-07-18/`	the 120 quarantined skill-leak dev runs (deviation 6 evidence)	`f89e7a41`
`bench/v2/`	v2 harness + `AGENT_MANUAL.md`	`c7629584`, `21032c06`
`bench/v2/data/`	MathlibQR/MPR (LeanSearch-v2 @`94f4888cbaf9`, CC BY 4.0), SorryDB-2601 split, `tasks_frozen.jsonl`	`c7629584`, `87638cfc`, `db679e8d`
`bench/v2/runs/`	every v2 run row **with full gzipped stream-json transcripts** (QR 810×4, MPR 69×4, SorryDB 508 rows) + `verify.jsonl` (197 kernel verdicts)	`3664aa3d`, `daac6107`, `382e51bf`, `1fe21cca`

Table 7: File map of the preservation repository.

`tasks_frozen.jsonl` ids, counting `gave_up`, `error`, verdicts, and `transcript_stats.cost_usd`). The figures in this document are generated from those same files by `report/bridge-report/recompute_figdata.py` (which asserts every plotted value against the markdown report) and `generate_figures.py`.

External sources cited (all fetched and verified during the v2 design sweep): TheoremGraph [1] · LeanSearch [2] · LeanSearch-v2 [3] · LeanExplore [4] · SorryDB [5] · FATE [6] · miniCTX [7] · LeanDojo [8] · LeanAgent [9] · Numina-Lean-Agent [10] · miniF2F-v2 [11] · ProofNet#/ProofNetVerif [12] · benchmark-faults survey [13].

References

[1] TheoremGraph. arXiv:2606.25363. `https://arxiv.org/abs/2606.25363`

[2] LeanSearch. arXiv:2403.13310. `https://arxiv.org/abs/2403.13310`

[3] LeanSearch-v2. arXiv:2605.13137. Data: pinned `frenzymath/LeanSearch-v2` @`94f4888cbaf9`, CC BY 4.0. `https://arxiv.org/abs/2605.13137`; `https://github.com/frenzymath/LeanSearch-v2`

[4] LeanExplore. arXiv:2506.11085. `https://arxiv.org/abs/2506.11085`

[5] SorryDB. arXiv:2603.02668. `https://arxiv.org/abs/2603.02668`; `https://sorrydb.org`

[6] FATE. arXiv:2511.02872. `https://arxiv.org/abs/2511.02872`

[7] miniCTX. arXiv:2408.03350. `https://arxiv.org/abs/2408.03350`

[8] LeanDojo. arXiv:2306.15626. `https://arxiv.org/abs/2306.15626`

[9] LeanAgent. arXiv:2410.06209. `https://arxiv.org/abs/2410.06209`

[10] Numina-Lean-Agent. arXiv:2601.14027. `https://arxiv.org/abs/2601.14027`

[11] miniF2F-v2. arXiv:2511.03108. `https://arxiv.org/abs/2511.03108`

[12] ProofNet#/ProofNetVerif. arXiv:2406.07222. Tasks pinned to `PAug/ProofNetSharp` `@a8da405f` (MIT). `https://arxiv.org/abs/2406.07222`

[13] Benchmark-faults survey. arXiv:2606.29493. `https://arxiv.org/abs/2606.29493`